



MCP Security Audit Report

AgentSign

notebooklm-mcp-cli

29 July 2026

Report ID: MCPS-MS5IMMB2

Executive Summary

FAIL

Risk Score

100/100

10
CRITICAL

0
HIGH

0
MEDIUM

0
LOW

SCAN SCOPE 195 files 53,952 lines	LANGUAGES json (1), python (194)	MCPS SDK Not Found
--	--	------------------------------

Risk Comparison: WITHOUT MCPS vs WITH MCPS

CURRENT EXPOSURE			WITH MCPS PROTECTION		
MCP-01	Rug Pulls Vulnerable pattern detected, no mitigation	●	MITIGATED Passport identity binding — tools are tied to verified agent identity <code>createPassport()</code> , <code>signPassport()</code>		●
MCP-03	Privilege Escalation via Tool Composition Vulnerable pattern detected, no mitigation	●	MITIGATED Tool integrity signing — each tool definition is cryptographically signed <code>signTool()</code> , <code>verifyTool()</code>		●
MCP-04	Cross-Server Request Forgery Vulnerable pattern detected, no mitigation	●	MITIGATED Signed tool definitions prevent tampering during cross-server calls <code>signTool()</code> , <code>tool_hash</code>		●
MCP-06	Indirect Prompt Injection via MCP MCPS mitigation detected	●	MITIGATED Signed message envelopes — all MCP messages carry cryptographic signatures <code>signMessage()</code> , <code>verifyMessage()</code>		●
MCP-07	Resource Exhaustion & DoS MCPS mitigation detected	●	MITIGATED Passport verification middleware — all requests require valid signed identity <code>secureMCP()</code> , <code>verifyPassport()</code>		●
MCP-08	Insufficient Logging & Audit Vulnerable pattern detected, no mitigation	●	MITIGATED Signed audit trail — every message exchange is logged with cryptographic proof <code>auditLog()</code> , <code>onAudit()</code>		●
MCP-09	Insecure MCP-to-MCP Communication MCPS mitigation detected	●	MITIGATED Origin binding — passports encode allowed origins, verified on each request <code>validateOrigin()</code> , <code>passport.origin</code>		●
MCP-10	Context Window Pollution MCPS mitigation detected	●	MITIGATED Envelope isolation — signed message boundaries prevent context pollution <code>signMessage()</code> , <code>envelope</code>		●
Risk Reduction					
100		-->	100		

OWASP MCP Top 10 Compliance

Coverage: 4/8 risks mitigated by MCPS

ID	Risk	Status	Current State	With MCPS
MCP-01	Rug Pulls	FAIL	Vulnerable pattern detected, no mitigation	Passport identity binding — tools are tied to verified agent identity
MCP-02	Tool Poisoning	N/A	Not applicable to code-level analysis	N/A — requires LLM-level defense
MCP-03	Privilege Escalation via Tool Composition	FAIL	Vulnerable pattern detected, no mitigation	Tool integrity signing — each tool definition is cryptographically signed
MCP-04	Cross-Server Request Forgery	FAIL	Vulnerable pattern detected, no mitigation	Signed tool definitions prevent tampering during cross-server calls
MCP-05	Sampling Manipulation	N/A	Not applicable to code-level analysis	N/A — requires client-level defense
MCP-06	Indirect Prompt Injection via MCP	PASS	MCPS mitigation detected	Signed message envelopes — all MCP messages carry cryptographic signatures
MCP-07	Resource Exhaustion & DoS	PASS	MCPS mitigation detected	Passport verification middleware — all requests require valid signed identity
MCP-08	Insufficient Logging & Audit	FAIL	Vulnerable pattern detected, no mitigation	Signed audit trail — every message exchange is logged with cryptographic proof
MCP-09	Insecure MCP-to-MCP Communication	PASS	MCPS mitigation detected	Origin binding — passports encode allowed origins, verified on each request
MCP-10	Context Window Pollution	PASS	MCPS mitigation detected	Envelope isolation — signed message boundaries prevent context pollution

Quick Fix `npm install mcp-secure`

Add MCPS to your MCP server to automatically mitigate the risks marked FAIL above.

OWASP Agentic AI Top 10 Coverage

12 rules checked | 10 findings

Rule	OWASP	MITRE	STRIDE	Severity	Findings	Status
AS-001	AA-03	T1059	Elevation of Privilege	CRITICAL	7	
AS-002	AA-05	T1552	Information Disclosure	HIGH	0	
AS-003	AA-04	T1078	Elevation of Privilege	MEDIUM	0	
AS-004	AA-02	T1190	Tampering	HIGH	0	
AS-005	AA-02	T1055	Tampering	CRITICAL	3	
AS-006	AA-09	T1610	Elevation of Privilege	HIGH	0	
AS-007	AA-06	T1195	Tampering	LOW	0	
AS-008	AA-01	T1548	Elevation of Privilege	HIGH	0	
AS-009	AA-07	T1059.007	Tampering	MEDIUM	0	
AS-010	AA-08	T1562.002	Repudiation	MEDIUM	0	
AS-011	AA-10	T1041	Information Disclosure	HIGH	0	
AS-012	MCP-07	T1078.004	Spoofing	HIGH	0	

Detailed Findings

#1 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[scripts/inject_cookies_and_inspect.py:107](#)

Dangerous execution: (async function) {

```
105 |         {
106 |             "expression": ""
> 107 |         (async function() {
108 |             function sleep(ms) { return new Promise(resolve => setTimeout(resolve, ms)); }...
109 |     }
```

Remediation:

1. Replace exec()/eval() with AST-parsed execution (ast.literal_eval for data, RestrictedPython for code). 2. Run agent code in Docker/gVisor sandbox with --no-new-privileges. 3. Use allowlist of permitted modules via importlib. 4. Add mandatory human confirmation before shell commands.

#2 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[scripts/inject_cookies_and_inspect.py:158](#)

Dangerous execution: (async function) {

```
156 |         {
157 |             "expression": ""
> 158 |         (async function() {
159 |             function sleep(ms) { return new Promise(resolve => setTimeout(resolve, ms)); }
160 |     }
```

Remediation:

1. Replace exec()/eval() with AST-parsed execution (ast.literal_eval for data, RestrictedPython for code). 2. Run agent code in Docker/gVisor sandbox with --no-new-privileges. 3. Use allowlist of permitted modules via importlib. 4. Add mandatory human confirmation before shell commands.

#3 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[scripts/inspect_upload_dom.py:104](#)

Dangerous execution: (function) {

```
102 |         {
103 |             "expression": ""
> 104 |         (function() {
105 |             const walker = document.createTreeWalker(document.body, NodeFilter.SHOW_TEXT);
106 |             const matches = [];
```

Remediation:

1. Replace exec()/eval() with AST-parsed execution (ast.literal_eval for data, RestrictedPython for code). 2. Run agent code in Docker/gVisor sandbox with --no-new-privileges. 3. Use allowlist of permitted modules via importlib. 4. Add mandatory human confirmation before shell commands.

#4 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[src/notebooklm_tools/cli/commands/doctor.py:385](#)

Dangerous execution: result = subprocess.run(# nosec B603 — cmd from shutil.which(), args are hardc

```
383 |         if claude_cmd:
384 |             try:
> 385 |                 result = subprocess.run( # nosec B603 — cmd from shutil.which(), args are
hardcoded strings, no shell=True
386 |                 [claude_cmd, "mcp", "list"],...
387 |                 capture_output=True,
```

Remediation:

1. Replace exec()/eval() with AST-parsed execution (ast.literal_eval for data, RestrictedPython for code). 2. Run agent code in Docker/gVisor sandbox with --no-new-privileges. 3. Use allowlist of permitted modules via importlib. 4. Add mandatory human confirmation before shell commands.

#5 AS-005 -- INJECTION PATTERN

CRITICAL

OWASP: AA-02 | MITRE: T1055 | STRIDE: Tampering

Known prompt injection, SQL injection, XSS, or command injection patterns

[src/notebooklm_tools/core/download.py:1103](#)

Known injection pattern detected

```
1101 |         Tries multiple extraction patterns for robustness:
1102 |         1. data-app-data attribute (primary)
> 1103 |         2. <script id="application-data"> tag (fallback)
1104 |         3. Other common patterns
1105 |
```

Remediation:

1. Validate and sanitize all inputs with strict allowlists. 2. Use parameterized queries for any database access (never string concatenation). 3. HTML-encode all agent output before rendering. 4. Implement content security policy (CSP) headers. 5. Block known injection patterns at the input boundary.

#6 AS-005 -- INJECTION PATTERN

CRITICAL

OWASP: AA-02 | MITRE: T1055 | STRIDE: Tampering

Known prompt injection, SQL injection, XSS, or command injection patterns

[src/notebooklm_tools/core/download.py:1131](#)

Known injection pattern detected

```
1129 |         logger.debug(f"JSON preview: {decoded_json[:200]}...")
1130 |
> 1131 |         # Pattern 2: <script id="application-data"> tag
1132 |         match = re.search(
1133 |             r'<script[^>]+id=["\']application-data["\'][^>]*>(.*?)</script>',
```

Remediation:

1. Validate and sanitize all inputs with strict allowlists. 2. Use parameterized queries for any database access (never string concatenation). 3. HTML-encode all agent output before rendering. 4. Implement content security policy (CSP) headers. 5. Block known injection patterns at the input boundary.

#7 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[tests/services/test_auth_health.py:331](#)

Dangerous execution: usable, status, detail = __import__(

```
329 |         lambda: type("C", (), {"check": lambda self, **kw: report})(),
330 |         )
> 331 |         usable, status, detail = __import__(
332 |             "notebooklm_tools.services.auth", fromlist=["credentials_are_usable"]
333 |         ).credentials_are_usable()
```

Remediation:

1. Replace `exec()/eval()` with AST-parsed execution (`ast.literal_eval` for data, `RestrictedPython` for code). 2. Run agent code in Docker/gVisor sandbox with `--no-new-privileges`. 3. Use allowlist of permitted modules via `importlib`. 4. Add mandatory human confirmation before shell commands.

#8 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[tests/services/test_auth_service.py:153](#)

Dangerous execution: `exec("from notebooklm_tools.services.auth import AuthManager", local_namespace)`

```
151 |         # Re-execute the import statement the same way cli code does.
> 152 |         local_namespace = {}
    153 |         exec("from notebooklm_tools.services.auth import AuthManager", local_namespace)
    154 |         assert local_namespace["AuthManager"] is sentinel_class...
    155 |
```

Remediation:

1. Replace `exec()/eval()` with AST-parsed execution (`ast.literal_eval` for data, `RestrictedPython` for code). 2. Run agent code in Docker/gVisor sandbox with `--no-new-privileges`. 3. Use allowlist of permitted modules via `importlib`. 4. Add mandatory human confirmation before shell commands.

#9 AS-005 -- INJECTION PATTERN

CRITICAL

OWASP: AA-02 | MITRE: T1055 | STRIDE: Tampering

Known prompt injection, SQL injection, XSS, or command injection patterns

[tests/test_api_client.py:119](#)

Known injection pattern detected

```
117 |         # Simplified HTML to minimize whitespace issues
> 118 |         # api_client.py looks for "FdrFJe":"..." for the session ID
    119 |         html = '<html><script>WIZ_global_data = {"SNlM0e":"new_csrf_token", "FdrFJe":"123456789"};</script></html>'
    120 |
    121 |         # Restore the original method for this test
```

Remediation:

1. Validate and sanitize all inputs with strict allowlists. 2. Use parameterized queries for any database access (never string concatenation). 3. HTML-encode all agent output before rendering. 4. Implement content security policy (CSP) headers. 5. Block known injection patterns at the input boundary.

#10 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[tests/test_io_encoding_windows.py:37](#)

Dangerous execution: `with __import__("pytest").raises(UnicodeEncodeError):`

```
35 |
> 36 |     stream = io.TextIOWrapper(io.BytesIO(), encoding="cp1252", errors="strict")
    37 |     with __import__("pytest").raises(UnicodeEncodeError):
    38 |         stream.write("\u2192")
    39 |
```

Remediation:

1. Replace `exec()/eval()` with AST-parsed execution (`ast.literal_eval` for data, `RestrictedPython` for code). 2. Run agent code in Docker/gVisor sandbox with `--no-new-privileges`. 3. Use allowlist of permitted modules via `importlib`. 4. Add mandatory human confirmation before shell commands.

Remediation Checklist

Prioritized by severity. Address CRITICAL items first.

Quick Win

`npm install mcp-secure`

Adds cryptographic passports, signed messages, tool verification, and audit logging.

CRITICAL (2)

- ☐ **AS-001: unsafe execution**
Replace `exec()/eval()` with AST-parsed execution (`ast.literal_eval` for data, `RestrictedPython` for code).
- ☐ **AS-005: injection pattern**
Validate and sanitize all inputs with strict allowlists.

Methodology & Standards

Scanner	mcps-audit v1.0.0
Rules Version	1.1.0
Scanned At	29/07/2026, 10:18:54
Report ID	MCPS-MS5IMMB2

Referenced Standards

- OWASP MCP Top 10**
Security risks for Model Context Protocol servers and clients.
owasp.org/www-project-mcp-top-10
- OWASP Agentic AI Top 10**
Security risks for autonomous AI agents.
owasp.org/www-project-agentic-ai-top-10
- MCPS -- IETF Internet-Draft**
Cryptographic security layer for MCP: draft-sharif-mcps-secure-mcp.
datatracker.ietf.org/doc/draft-sharif-mcps-secure-mcp
- MITRE ATT&CK**
Adversary tactics and techniques knowledge base.
attack.mitre.org
- STRIDE Threat Model**
Microsoft threat classification framework.
microsoft.com/en-us/security/blog/stride

AgentSign
Cryptographic identity and security for AI agents
agentsign.dev